fn make_private_group_by

Michael Shoemate

This proof resides in **"contrib"** because it has not completed the vetting process.

Proves soundness of `fn make_private_group_by` in `mod.rs` at commit 0db9c6036 (outdated[1]).

1 Hoare Triple

Precondition

Compiler-verified

- Generic `MS` must implement trait `DatasetMetric`.
- Generic `MI` must implement trait `UnboundedMetric`.
- Generic `MO` must implement trait `ApproximateMeasure`.

User-verified

None

Pseudocode

```
def make_private_group_by(
    input_domain,
    input_metric,
    output_measure,
    plan,
    global_scale,
    threshold,
):
    input, keys, aggs, key_sanitizer = match_group_by(plan) #

    # 1: establish stability of input
    t_prior = input.make_stable(input_domain, input_metric)  #
    middle_domain, middle_metric = t_prior.output_space()

    by = match_grouping_columns(keys)  #
    margin = middle_domain.get_margin(by, Margin.default())  #

    match key_sanitizer:
        case KeySanitizer.Join(labels):
            num_keys = LazyFrame.from_(labels).select([len()]).collect()
            margin.max_num_partitions = num_keys.column("len").u32().last()  #

```

[1] See new changes with `git diff 0db9c6036..0c9b0bd4 rust/src/measurements/make_private_lazyframe/group_by/mod.rs`

```
            is_join = True
        case _:
            is_join = False

    # 2: prepare for release of `aggs`
    expr_domain = WildExprDomain(  #
        columns=middle_domain.series_domains,
        context=ExprContext.Grouping(by, margin),
    )

    m_exprs = make_basic_composition([
        make_private_expr(
            expr_domain,
            PartitionDistance(middle_metric),
            output_measure,
            expr,
            global_scale,
        ) for expr in aggs
    ])

    f_comp = m_exprs.function
    f_privacy_map = m_exprs.privacy_map

    # 3: prepare for release of `keys`
    dp_exprs, null_exprs = zip(*((ep.expr, ep.fill) for ep in m_exprs.invoke(input)))

    # 3.1: reconcile information about the threshold
    if margin.public_info is not None or is_join:  #
        threshold_info = None
    elif match_filter(key_sanitizer) is not None:  #
        name, threshold_value = match_filter(key_sanitizer)
        noise = find_len_expr(dp_exprs, name)[1]
        threshold_info = name, noise, threshold_value, False
    elif threshold is not None:  #
        name, noise = find_len_expr(dp_exprs, None)
        threshold_info = name, noise, threshold_value, True
    else:  #
        raise f"The key set of {by} is private and cannot be released."

    # 3.2: update key sanitizer
    if threshold_info is not None:  #
        name, _, threshold_value, is_present = threshold_info
        threshold_expr = col(name).gt(lit(threshold_value))
        if not is_present and predicate is not None:  #
            key_sanitizer = KeySanitizer.Filter(threshold_expr.and_(predicate))
        else:
            key_sanitizer = KeySanitizer.Filter(threshold_expr)

    elif isinstance(key_sanitizer, KeySanitizer.Join):  #
        key_sanitizer.fill_null = []
        for dp_expr, null_expr in zip(dp_exprs, null_exprs):
            name = dp_expr.meta().output_name()
            if null_expr is None:
                raise f"fill expression for {name} is unknown"

            key_sanitizer.fill_null.append(col(name).fill_null(null_expr))

    # 4: build final measurement
    def function(arg: DslPlan) -> DslPlan:  #
        output = DslPlan.GroupBy(
            input=arg,
            keys=keys,
            aggs=[p.expr for p in f_comp.eval(arg)],
            apply=None,
```

```
            maintain_order=False,
        )
        match key_sanitizer:
            case KeySanitizer.Filter(predicate):
                output = DslPlan.Filter(input=output, predicate=predicate)
            case KeySanitizer.Join(
                labels,
                how,
                left_on,
                right_on,
                options,
                fill_null,
            ):
                match how:  #
                    case JoinType.Left:
                        input_left, input_right = labels, output
                    case JoinType.Right:
                        input_left, input_right = output, labels
                    case _:
                        raise "unreachable"

                output = DslPlan.HStack(
                    input=DslPlan.Join(
                        input_left,
                        input_right,
                        left_on,
                        right_on,
                        options,
                        predicates=[],
                    ),
                    exprs=fill_null,
                    options=ProjectionOptions.default(),
                )
        return output

    def privacy_map(d_in):  #
        mip = margin.get("max_influenced_partitions", default=d_in)
        mnp = margin.get("max_num_partitions", default=d_in)
        mpc = margin.get("max_partition_contributions", default=d_in)
        mpl = margin.get("max_partition_length", default=d_in)

        l0 = min(mip, mnp, d_in)
        li = min(mpc, mpl, d_in)
        l1 = l0.inf_mul(li).min(d_in)

        d_out = f_privacy_map.eval((l0, l1, li))

        if margin.public_info is None or is_join:  #
            pass
        elif threshold is not None:  #
            _, noise, threshold_value = threshold_info
            if li >= threshold_value:
                raise f"Threshold must be greater than {li}."
            d_instability = threshold_value.inf_sub(li)
            delta_single = integrate_discrete_noise_tail(
                noise.distribution, noise.scale, d_instability
            )
            delta_joint = 1 - (1 - delta_single).inf_powi(l0)
            d_out = MO.add_delta(d_out, delta_joint)
        else:
            raise "keys must be public if threshold is unknown"

        return d_out

```

```
    m_group_by_agg = Measurement.new(
        middle_domain,
        function,
        middle_metric,
        output_measure,
        privacy_map,
    )

    return t_prior >> m_group_by_agg
```

Postconditions

Theorem 1.1.

Theorem 1.2. For every setting of the input parameters `(input_domain, input_metric, output_measure, plan, global_scale, threshold, MS, MI, MO)` to `make_private_group_by` such that the given preconditions hold, `make_private_group_by` raises an exception (at compile time or run time) or returns a valid measurement. A valid measurement has the following property:

1. (Privacy guarantee). For every pair of elements x, x' in `input_domain` and for every pair (`d_in`, `d_out`), where `d_in` has the associated type for `input_metric` and `d_out` has the associated type for `output_measure`, if x, x' are `d_in`-close under `input_metric`, `privacy_map`(`d_in`) does not raise an exception, and `privacy_map`(`d_in`) $\leq$ `d_out`, then `function`(x), `function`(x') are `d_out`-close under `output_measure`.

2 Proof

We now prove the postcondition (Theorem 1.1).

Proof. The function logic breaks down into parts:

1. establish stability of input (line 11)
2. prepare for release of `aggs` (line 27)
3. prepare for release of `keys` (line 46)
   (a) reconcile information about the threshold (line 49)
   (b) update key sanitizer (line 62)
4. build final measurement (line 80)
   (a) construct function (line 81)
   (b) construct privacy map (line 122)

`match_group_by` returns a valid `MatchGroupBy` struct. In this struct, `input` is the input plan, `keys` is the grouping keys, `aggs` is the list of expressions to compute per-partition, and `key_sanitizer` details how to sanitize the key-set (line 9).

2.1 Stability of input

Start by establishing properties of the following variables, which hold for any setting of the input arguments.

By the postcondition of `StableDslPlan.make_stable`, `t_prior` is a valid transformation (line 12). By the postcondition of `match_grouping_columns`, `grouping_columns` holds the names of the grouping columns. `margin` denotes what is considered public information about the key set, pulled from descriptors in the input domain (line 15).

When sanitizing keys via a join, an upper bound on the total number of partitions can be statically derived via the length of the grouping keys. Line 21 retrieves this information from the key-set and assigns it to the margin.

`is_join` indicates that key sanitization will occur via a join. This will be a useful criteria later.

2.2 Prepare to release `aggs`

Line 28 starts the process to prepare a joint measurement for releasing the per-partition aggregations `aggs`. Each measurement's input domain is the wildcard expression domain, used to prepare computations that will be applied over data grouped by grouping columns `by`.

By the postcondition of `make_basic_composition`, `m_exprs` is a valid measurement that prepares a batch of expressions that, when executed via `f_comp`, satisfies the privacy guarantee of `f_privacy_map`.

Now that we've prepared the necessary prerequisites for privatizing the aggregations, we switch to privatizing the keys.

2.3 Prepare to release `keys`

`key_sanitization` needs to be updated with information that was not available in the initial match on line 9.

- When filtering, a threshold may be passed into the constructor, and we must determine a suitable column to filter/threshold against.
- When joining, we need expressions for filling null values corresponding to partitions that don't exist in the sensitive data.

We first reconcile information about the filtering threshold (line 49).

- In the setting where grouping keys are considered public, or key sanitization is handled via a join, no thresholding is necessary (line 50).
- Otherwise, if the key sanitizer contains filtering criteria (line 52), then by the postcondition of `find_len_expr`, filtering on `name` can be used to satisfy δ-approximate DP. `noise` of type `NoisePlugin` details the noise distribution and scale. `threshold_info` then contains the column name, noise distribution, threshold value and whether a filter needs to be inserted into the query plan. In this case, since the threshold comes from the query plan, it is not necessary to add it to the query plan, and is therefore false.
- In the case that a threshold has been provided to the constructor (line 56), then `find_len_expr` will search for a suitable column to threshold on, returning with the `name` and `noise` distribution of the column. Since the threshold comes from the constructor and not the plan, it will be necessary to add this filtering threshold to the query plan (explaining the true value).
- By line 59 no suitable filtering criteria have been found, and by the first case there is no suitable invariant for the margin or explicit join keys, so it is not possible to release the keys in a way that satisfies differential privacy, and the constructor refuses to build a measurement.

In common use through the context API, if a mechanism is allotted a delta parameter for stable key release but doesn't already satisfy approximate-DP, then a search is conducted for the smallest suitable threshold parameter. The branching logic from line 49 is intentionally written to ignore the constructor threshold when a suitable filtering threshold is already detected in the plan, to avoid overwriting/changing it.

We now update `key_sanitization` starting from line 62:

- When filtering (line 63), `threshold_info` will always be set. `threshold_expr` reflects the reconciled criteria, using the chosen filtering column and threshold. This threshold expression is applied either way the logic branches on line 66. The first case preserves any additional filtering criteria that was already present in the plan, but not used for key release.

- When joining (line 71) the sanitizer needs a way to fill missing values from partitions missing in the data. This is provided by `null_exprs`, which contain imputation strategies for filling in missing values in a way that is indistinguishable from running the mechanism on an empty partition.

`key_sanitizer` now contains all necessary information to ensure that the keys are sanitized, and will be used to construct the function. `threshold_info`, `is_join` and `public_info` are consistent with `key_sanitizer`, and will be used to construct the privacy map.

2.4 Build final measurement

We now move on to the implementation of the function on line 81, using all of the properties shown of the variables established thus far. The function returns a DslPlan that applies each expression from `m_exprs` to `arg` grouped by `keys`. `key_sanitizer` is conveyed into the plan, if set, to ensure that the keys are also privatized if necessary.

In the case of the join privatization, by the definition of `KeySanitizer`, the join will either be a left or right join. The branching swaps the input plan and labels plan to ensure that the sensitive input data is always joined against the labels, but using the same join type as in the original plan. Once the join is applied, the fill imputation expressions are applied, hiding which partitions don't exist in the original data.

It is assumed that the emitted DSL is executed in the same fashion as is done by Polars. This proof/implementation does not take into consideration side-channels involved in the execution of the DSL.

We now move on to the implementation of the privacy map. 122 The measurement for each expression expects data set distances in terms of a triple:

- L^0: the greatest number of partitions that can be influenced by any one individual. This is no greater than the input distance (an individual can only ever influence as many partitions as they contribute rows), but could be smaller when supplemented by the `max_influenced_partitions` metric descriptor or `max_num_partitions` domain descriptor.

- L^∞: the greatest number of records that can be added or removed by any one individual in each partition. This is no greater than the input distance, but could be tighter when supplemented by the `max_partition_contributions` metric descriptor or the `max_partition_length` domain descriptor.

- L^1: the greatest total number of records that can be added or removed across all partitions. This is no greater than the input distance, but could be tighter when accounting for the L^0 and L^∞ distances.

By the postcondition of `f_privacy_map`, the privacy loss of releasing the output of `aggs`, when grouped data sets may differ by this distance triple, is `d_out`.

We also need to consider the privacy loss from releasing `keys`. On line 134 under the `public_info` invariant, or under the join sanitization, releases on any neighboring datasets x and x' will share the same key-set, resulting in zero privacy loss.

We now adapt the proof from [Rog23] (Theorem 7) to consider the case of stable key release from line 136. Consider S to be the set of labels that are common between x and x'. Define event E to be any potential outcome of the mechanism for which all labels are in S (where only stable partitions are released). We then lower bound the probability of the mechanism returning an event E. In the following, c_j denotes the exact count for partition j, and Z_j is a random variable distributed according to the distribution used to release a noisy count.

$$
\begin{aligned}
\Pr[E] &= \prod_{j \in x \setminus x'} \Pr[c_j + Z_j \leq T] \\
&\geq \prod_{j \in x \setminus x'} \Pr[\Delta_\infty + Z_j \leq T] \\
&\geq \Pr[\Delta_\infty + Z_j \leq T]^{\Delta_0}
\end{aligned}
$$

The probability of returning a set of stable partitions ($\Pr[E]$) is the probability of not returning any of the unstable partitions. We now solve for the choice of threshold T such that $\Pr[E] \geq 1 - \delta$.

$$
\begin{aligned}
\Pr[\Delta_\infty + Z_j \leq T]^{\Delta_0} &= \Pr[Z_j \leq T - \Delta_\infty]^{\Delta_0} \\
&= (1 - \Pr[Z_j > T - \Delta_\infty])^{\Delta_0}
\end{aligned}
$$

Let `d_instability` denote the distance to instability of $T - \Delta_\infty$. By the postcondition of `integrate_discrete_noise_tail`, the probability that a random noise sample exceeds `d_instability` is at most `delta_single`. Therefore $\delta = 1 - (1 - \texttt{delta_single})^{\Delta_0}$. This privacy loss is then added to `d_out`.

Together with the potential increase in delta for the release of the key set, then it is shown that `function`(x), `function`(x') are `d_out`-close under `output_measure`.

□

References

[Rog23] Ryan Rogers. A unifying privacy analysis framework for unknown domain algorithms in differential privacy, 2023.